Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Autonomous AI Agents with OpenClaw

TGS Ref No: TGS-2026064859

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	July 2026	First version — step-by-step guide to all 10 OpenClaw labs: hosting, LLM models, channels, tools, skills, cron jobs, security, dashboard, cost saving and blockchain invoice verification	Tertiary Infotech Academy Pte Ltd
2.0	23 August 2026	Updated course title and code; aligned duration, assessment method, repository links, and all learner-facing metadata to Autonomous AI Agents with OpenClaw.	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Autonomous AI Agents with OpenClaw — Step-by-Step Learner Guide

Course Code: TGS-2026064859 · **Version 2.0** · Tertiary Infotech Academy Pte Ltd

Welcome! This guide walks you command-by-command through every hands-on lab. In one day you deploy your own AI agent platform, connect it to LLMs and messaging channels, add tools and skills, secure it, and verify financial documents with blockchain logic — all running on a Hostinger VPS or your local Docker Desktop.

Note: Work through the labs in order — each builds on the last. Whenever you see a **Test it** box, stop and confirm the result before moving on.

---

0. Before You Start

0.1 What you need

Requirement	Details	Where to get it
Laptop	Windows 10/11, macOS 12+, or Ubuntu 22.04+	Your own machine
Docker Desktop (Option B)	Required for local labs + Lab 10 blockchain	docker.com/products/docker-desktop
Hostinger VPS (Option A)	Ubuntu 22.04 LTS, SSH access, runs 24/7	https://www.hostinger.com?REFERRALCODE=FEGANGCHQ20C
Telegram account	Lab 3 — channel setup via BotFather	telegram.org
WhatsApp (phone)	Lab 3 — QR pairing	Pre-installed on your phone
Groq API key (free)	Lab 2 — no credit card needed	console.groq.com
AgentMail account (free)	Lab 4 — email tool	agentmail.to
Firecrawl account (free)	Lab 4 — web scraper tool	firecrawl.dev

0.2 Two ways to run every lab

Option A — Hostinger VPS (recommended for 24/7 deployment). SSH into your Ubuntu 22.04 server and run all commands there. OpenClaw stays running after class.

Hostinger referral (discount): https://www.hostinger.com?REFERRALCODE=FEGANGCHQ20C

Option B — Docker Desktop (local laptop). Run OpenClaw in a Docker container on your Windows, macOS, or Linux machine. Requires Docker Desktop to be open and running.

0.3 Lab index

Lab	Title	Time
Lab 1	OpenClaw Hosting	30 min

Lab 2	OpenClaw Model	30 min
Lab 3	OpenClaw Channel	30 min
Lab 4	OpenClaw Tools	30 min
Lab 5	OpenClaw Skills	20 min
Lab 6	Cron Jobs and Heartbeat	30 min
Lab 7	OpenClaw Security	30 min
Lab 8	OpenClaw Dashboard	20 min
Lab 9	OpenClaw Cost Saving	20 min
Lab 10	Blockchain Invoice Verification	30 min

Lab 1 — OpenClaw Hosting

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — [OPTION A — Hostinger VPS] 1/3 — SSH into your VPS

```
ssh root@YOUR_VPS_IP
# Replace YOUR_VPS_IP with your actual Hostinger VPS IP address
```

Step 2 — [OPTION A — Hostinger VPS] 2/3 — Install OpenClaw

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Step 3 — [OPTION A — Hostinger VPS] 3/3 — Install and start the daemon

```
openclaw daemon install
systemctl enable --now openclaw
openclaw gateway status   # should show: Gateway: running   Port: 18789
```

Step 4 — [OPTION B — Docker Desktop] 1/5 — Open Docker Desktop and confirm it is running

```
docker version
# You should see: Server: Docker Engine - Community
# If Docker Desktop is not open, launch it first and wait for 'Engine running'
```

Step 5 — [OPTION B — Docker Desktop] 2/5 — Pull the OpenClaw image from Docker Hub

```
docker pull openclaw/openclaw:latest
# This downloads ~500 MB. Wait for 'Status: Downloaded newer image'
```

Step 6 — [OPTION B — Docker Desktop] 3/5 — Run the OpenClaw container

```
docker run -d --name openclaw \
 -p 18789:18789 \
 -v openclaw-data:/root/.openclaw \
 openclaw/openclaw:latest
```

Step 7 — [OPTION B — Docker Desktop] 4/5 — Confirm the container is up

```
docker ps
# Expected output (truncated):
# CONTAINER ID   IMAGE                    STATUS        PORTS
# xxxxxxxxxxxx   openclaw/openclaw:latest   Up X seconds   0.0.0.0:18789->18789/tcp
```

Step 8 — [OPTION B — Docker Desktop] 5/5 — Run the onboarding wizard inside the container

```
docker exec -it openclaw openclaw onboard
```

Step 9 — [BOTH OPTIONS] Open OpenClaw in your browser

```
http://localhost:18789
# Open this URL in Chrome or Firefox on your laptop
```

Test it: openclaw gateway status → Expected: Gateway: running Port: 18789

Lab 2 — OpenClaw Model

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Option A — Groq (free tier, no credit card required)

```
# Visit console.groq.com → sign up free → API Keys → Create key
openclaw model set groq --api-key YOUR_GROQ_KEY --model llama-3.1-70b-versatile
```

Step 2 — Option B — OpenAI OAuth (step 1: launch browser flow)

```
openclaw model set openai --oauth   # copy the printed URL to your browser
```

Step 3 — Option B — OpenAI OAuth (steps 2-4: authorise and confirm)

```
# Browser: Sign in to OpenAI → click Allow
# Terminal: ✓ OpenAI OAuth token saved.
```

Step 4 — Option C — Ollama (local, zero API cost)

```
ollama launch openclaw
openclaw model set ollama --host http://localhost:11434 --model openclaw
```

Step 5 — Option D — Edit config file directly

```
nano ~/.openclaw/openclaw.json
# Set: provider, model, apiKey
```

Step 6 — Test the model connection

```
openclaw model test   # expect: Status: ✓ Connected
```

Test it: openclaw model test → Expected: Provider: groq Model: llama-3.1-70b-versatile Status: ✓ Connected

---

Lab 3 — OpenClaw Channel

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Telegram — open BotFather in Telegram app

```
# Open Telegram ⟶ search @BotFather ⟶ send /newbot
```

Step 2 — Telegram — enter bot name and username, copy token

```
# Enter: Alfred Agent  (name)
# Enter: alfred_agent_bot  (username)
# Copy the API token shown
```

Step 3 — Telegram — register and start channel

```
openclaw channel add telegram --token YOUR_BOT_TOKEN
openclaw channel start telegram
```

Step 4 — WhatsApp — add channel

```
openclaw channel add whatsapp
```

Step 5 — WhatsApp — scan QR code on your phone

```
# QR code appears in terminal
# WhatsApp ⟶ Linked Devices ⟶ Link a Device ⟶ scan
```

Step 6 — WhatsApp — start channel

```
openclaw channel start whatsapp
```

Test it: openclaw channel list → Expected: telegram running | whatsapp running

Lab 4 — OpenClaw Tools

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — AgentMail — sign up and get API key

```
# Visit https://agentmail.to → create free account → copy API key
```

Step 2 — AgentMail — add tool

```
openclaw tools add agentmail --api-key YOUR_AGENTMAIL_KEY
```

Step 3 — Firecrawl — sign up and get fc-... key

```
# Visit https://www.firecrawl.dev → create account → copy fc-... key
```

Step 4 — Firecrawl — add tool

```
openclaw tools add firecrawl --api-key fc-YOUR_KEY
```

Step 5 — Agent Browser — get key and add tool

```
# Visit https://agent-browser.dev → get API key
openclaw tools add agent-browser --api-key YOUR_KEY
```

Step 6 — Test tools in chat

```
# Send: 'Check my email'
# Send: 'Browse https://news.ycombinator.com'
# Send: 'Scrape https://openclaw.ai'
```

Test it: `openclaw tools list` → Expected: agentmail enabled | firecrawl enabled | agent-browser enabled

Lab 5 — OpenClaw Skills

Estimated time: 20 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Browse ClawHub marketplace

```
# Visit https://clawhub.ai → explore available skills
```

Step 2 — Install web-research skill (VPS / local)

```
npx clawhub@latest install web-research
```

Step 3 — Install daily-briefing skill (Docker)

```
docker exec openclaw npx clawhub@latest install daily-briefing
```

Step 4 — List installed skills

```
openclaw skills list
```

Step 5 — Test skill in Telegram chat

```
# Send: /web-research What is OpenClaw?
```

Step 6 — Update a skill

```
npx clawhub@latest update web-research
```

Test it: `openclaw skills list` → Expected: web-research active | daily-briefing active

---

Lab 6 — Cron Jobs and Heartbeat

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Create a daily morning summary cron

```
openclaw cron create --schedule '0 9 * * *' \
  --prompt 'Summarise my email' --channel telegram --name morning
```

Step 2 — Create a weekly AI news cron

```
openclaw cron create --schedule '0 8 * * 1' \
  --prompt '/web-research Latest AI news' --channel telegram --name weekly
```

Step 3 — List crons and run one immediately

```
openclaw cron list
openclaw cron run morning
```

Step 4 — Configure HEARTBEAT.md

```
nano ~/.openclaw/HEARTBEAT.md
# interval: 30m
# channel: telegram
# message: OpenClaw is alive
```

Step 5 — Enable heartbeat via CLI

```
openclaw heartbeat enable --interval 30m --channel telegram
```

Step 6 — Check heartbeat status

```
openclaw heartbeat status
```

Test it: openclaw heartbeat status → Expected: Heartbeat: enabled Interval: 30m Channel: telegram

---

Lab 7 — OpenClaw Security

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Step 1 — Store API keys as environment variables

```
export GROQ_API_KEY='gsk_...'
echo 'export GROQ_API_KEY=...' >> ~/.bashrc
```

Step 2 — Step 2 — Set Telegram allowlist

```
openclaw channel config telegram --allowlist-add YOUR_TELEGRAM_USER_ID
```

Step 3 — Step 3 — Set gateway authentication token

```
openclaw config set gateway.auth-token $(openssl rand -hex 32)
```

Step 4 — Step 4 — Enable execution sandbox

```
mkdir ~/openclaw-sandbox
openclaw tools config exec --sandbox-dir ~/openclaw-sandbox
```

Step 5 — Step 5 — Disable unused tools

```
openclaw tools disable exec
```

Step 6 — Step 6 — Export and review audit logs

```
openclaw logs --last 50
openclaw logs --export ~/audit.json
```

Test it: `openclaw config list | grep -E 'gateway|auth'` → Expected: gateway.auth-token set | gateway.allowlist enabled

---

Lab 8 — OpenClaw Dashboard

Estimated time: 20 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Launch dashboard on local machine

```
openclaw dashboard   # opens http://localhost:18789
```

Step 2 — VPS — create SSH port-forward tunnel

```
ssh -L 18789:localhost:18789 root@YOUR_VPS_IP   # keep terminal open
```

Step 3 — VPS — open dashboard in your local browser

```
# Open: http://localhost:18789 in a NEW browser tab on your LOCAL machine
```

Step 4 — Explore: Overview and Channels panels

```
# Check: gateway status, uptime, model provider, connected channels
```

Step 5 — Explore: Memory panel

```
# View MEMORY.md, SOUL.md, AGENTS.md, HEARTBEAT.md
```

Step 6 — Restart gateway from the dashboard

```
# Dashboard → Gateway → Restart → wait 5 seconds → refresh page
```

Test it: `openclaw gateway status` → Expected: Gateway: running Port: 18789 Dashboard: http://localhost:18789

Lab 9 — OpenClaw Cost Saving

Estimated time: 20 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference:
https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu

Steps

Step 1 — Switch to cheapest Claude model (Haiku)

```
openclaw model set anthropic --model claude-haiku-4-5-20 251 001
```

Step 2 — Use Claude.ai subscription (fixed monthly fee, unlimited)

```
openclaw model set anthropic --subscription
```

Step 3 — Use MiniMax free tier (zero API cost)

```
openclaw model set minimax --api-key KEY --model abab6.5s-chat
```

Step 4 — Use Ollama locally (zero API cost, runs on your machine)

```
openclaw model set ollama --model openclaw
```

Step 5 — Enable context compaction to reduce token usage

```
openclaw config set context.compaction enabled
openclaw config set context.compaction-threshold 50 000
```

Step 6 — Set a monthly spending alert

```
openclaw usage stats
openclaw usage alert --threshold 5.00 --channel telegram
```

Test it: openclaw usage stats → Expected: Total spend this month: $0.00 | Compaction: enabled

---

Lab 10 — Blockchain Invoice Verification

Estimated time: 30 minutes · **Environment:** Hostinger VPS OR Docker Desktop

Lab reference: https://github.com/tertiarycourses/TGS-2026064859-Autonomous-AI-Agents-with-OpenClaw/tree/main/labs/lab-10-blockchain-invoice

Steps

Step 1 — Clone the Lab 10 source from GitHub

```
git clone https://github.com/tertiarycourses/TGS-2 026 064 859-Autonomous-AI-Agents-with-OpenClaw.git
cd TGS-2 026 064 859-Autonomous-AI-Agents-with-OpenClaw/labs/lab-10-blockchain-invoice
```

Step 2 — Build the Docker image

```
docker build -t openclaw/invoice-verify:latest .
```

Step 3 — Run the blockchain verification container

```
docker run -d -p 5000:5000 --name invoice-verify \
  openclaw/invoice-verify:latest
curl http://localhost:5000/health
```

Step 4 — Register a test invoice

```
curl -X POST http://localhost:5000/register \
  -H 'Content-Type: application/json' \
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
      "amount":1500.00,"date":"2026-07-11"}'
```

Step 5 — Verify the invoice — same data (should pass)

```
curl -X POST http://localhost:5000/verify \
  -H 'Content-Type: application/json' \
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
      "amount":1500.00,"date":"2026-07-11"}'
```

Step 6 — Tamper test — change amount (should fail)

```
curl -X POST http://localhost:5000/verify \
  -H 'Content-Type: application/json' \
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
      "amount":9999.00,"date":"2026-07-11"}'
# Expected: "verified": false, "tampered": true
```

Test it: curl http://localhost:5000/health → Expected: {"invoices": 1, "status": "ok"}

Reference Documentation

Resource	URL
OpenClaw Install	https://docs.openclaw.ai/install
LLM Providers	https://docs.openclaw.ai/providers
Channels	https://docs.openclaw.ai/channels
Tools	https://docs.openclaw.ai/tools
Skills / ClawHub	https://clawhub.ai
Security Guide	https://docs.openclaw.ai/gateway/security
Dashboard	https://docs.openclaw.ai/dashboard
Groq (free LLM API)	https://console.groq.com
Ollama (local model)	https://ollama.com
AgentMail	https://agentmail.to
Firecrawl	https://www.firecrawl.dev
Agent Browser	https://agent-browser.dev
Hostinger VPS	https://www.hostinger.com?REFERRALCODE=FEGANGCHQ20C
Course LMS	https://ai-lms-tms.tertiaryinfo.tech/

Final Assessment

Written Assessment (Short Answer) + Case Study — Open Book. The two instruments take 1 hour in total and assess knowledge plus application to one coherent business scenario.

- `openclaw gateway status` → Gateway: running
- Agent responds to a message via Telegram and/or WhatsApp
- Firecrawl tool returns a webpage summary
- Cron job fires on schedule and sends result to channel
- OpenClaw dashboard accessible at http://localhost:18789
- Blockchain: invoice INV-2026-001 registers and verifies via Docker on port 5000

Note: TRAQOM Survey and Certificate Delivery are mandatory at end of class. The trainer will display a QR code — scan it with your phone before leaving.